Лекция 3

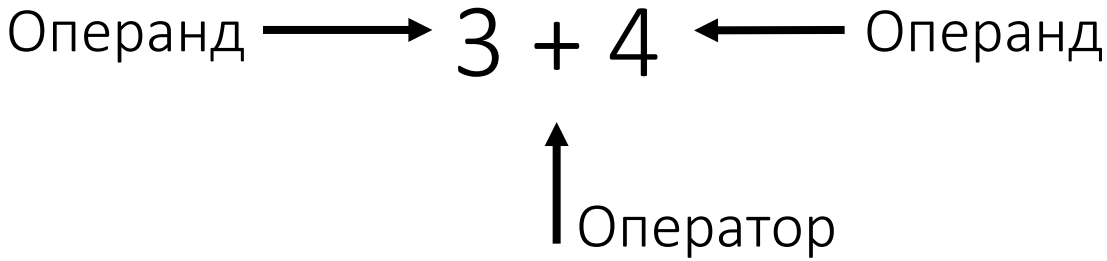
Функции работы с переменными в PHP. Математические операции.

Операторы и операнды

Для осуществления операций с переменными существуют различные группы операторов.

Оператор - это элемент программного кода, который описывает то или иное действие в выражении. В PHP оператор представляет собой символ, благодаря которому могут производиться различные виды вычислений, сравнений или присваиваний с участием одного или нескольких значений.

Операнд - это то, на что воздействует оператор. Операнды и операторы дают в результате выражение, которое формирует новое значение.

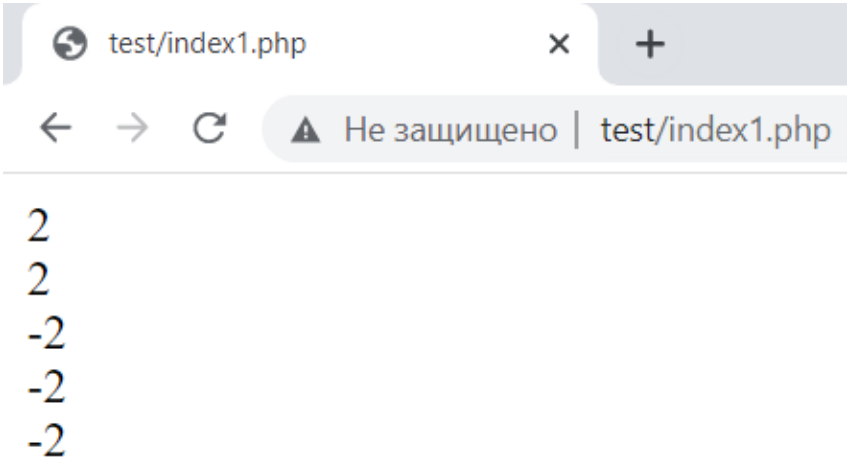


Арифметические операции в PHP

Пример	Название	Результат
-$a	Отрицание	Смена знака $a
$a + $b	Сложение	Сумма $a и $b
$a - $b	Вычитание	Разность $a и $b
$a * $b	Умножение	Произведение $a и $b
$a / $b	Деление	Частное от деления $a на $b
$a % $b	Деление по модулю	Целочисленный остаток от деления $a на $b
$a ** $b	Возведение в степень	Возводит значение переменной $a в степень заданную значением переменной $b

Арифметические операции в PHP

```php
<?php
echo (5 % 3)."<br>";          // выводит 2
echo (5 % -3)."<br>";         // выводит 2
echo (-5 % 3)."<br>";         // выводит -2
echo (-5 % -3)."<br>";        // выводит -2
echo (-5.5 % -3.3)."<br>";    // выводит -2
?>
```



Арифметические операции в PHP

1.Операция деления ("/"), начиная с версии 7, при делении целых чисел возможен результат типа как int, так и float.

2.Операцию вычисления остатка от деления "**%**" рекомендуется использовать только с целыми числами, так как применение ее к дробным может привести к нежелательному и непредсказуемому результату.

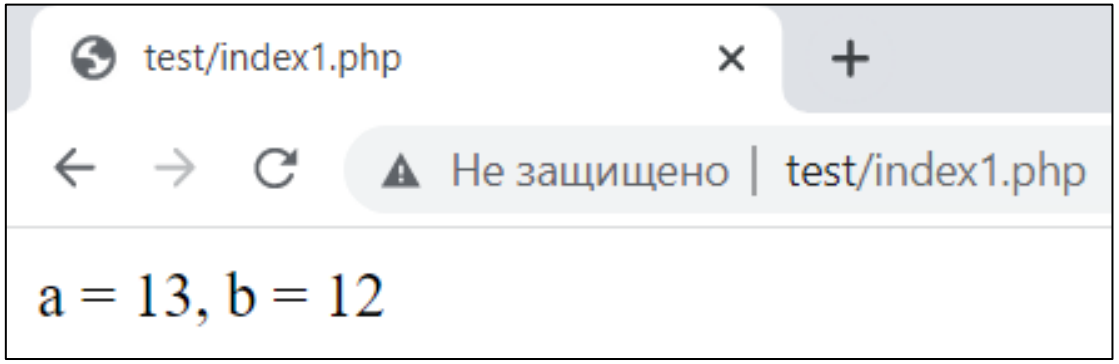
3.Приоритет одних математических операций над другими и изменение приоритетов при использовании скобок в арифметических выражениях соответствуют обычным математическим правилам.

Операции инкремента и декремента

Пример	Название	Действие
++$a	Префиксный инкремент	Увеличивает $a на единицу и возвращает значение $a
$a++	Постфиксный инкремент	Возвращает значение $a, а затем увеличивает $a на единицу
--$a	Префиксный декремент	Уменьшает $a на единицу и возвращает значение $a
$a--	Постфиксный декремент	Возвращает значение $a, а затем уменьшает $a на единицу

Операции инкремента и декремента

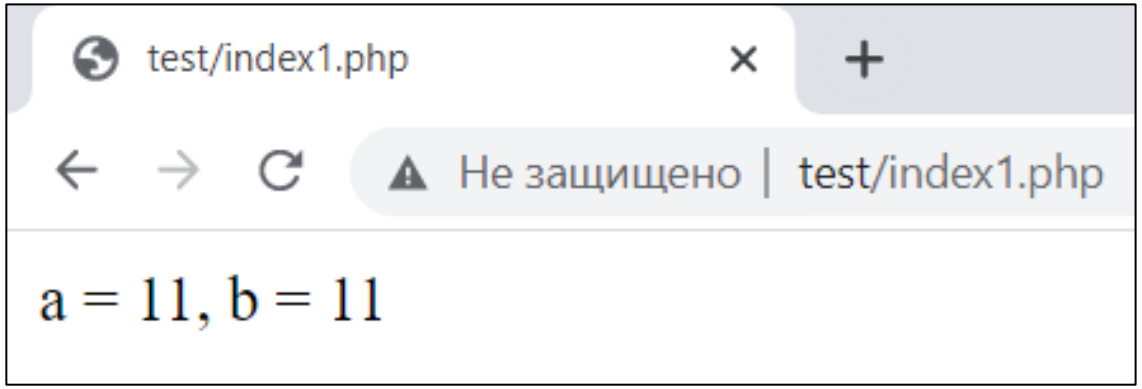
```php
$a = 12;
$b = $a++; // $b равно 12
echo "a = $a   b = $b";
```



Здесь сначала значение переменной $a передается переменной $b, а затем происходило увеличение значения переменной $a.

Операции инкремента и декремента

```php
$a = 12;
$b = --$a; // $b равно 11
echo "a = $a   b = $b";
```



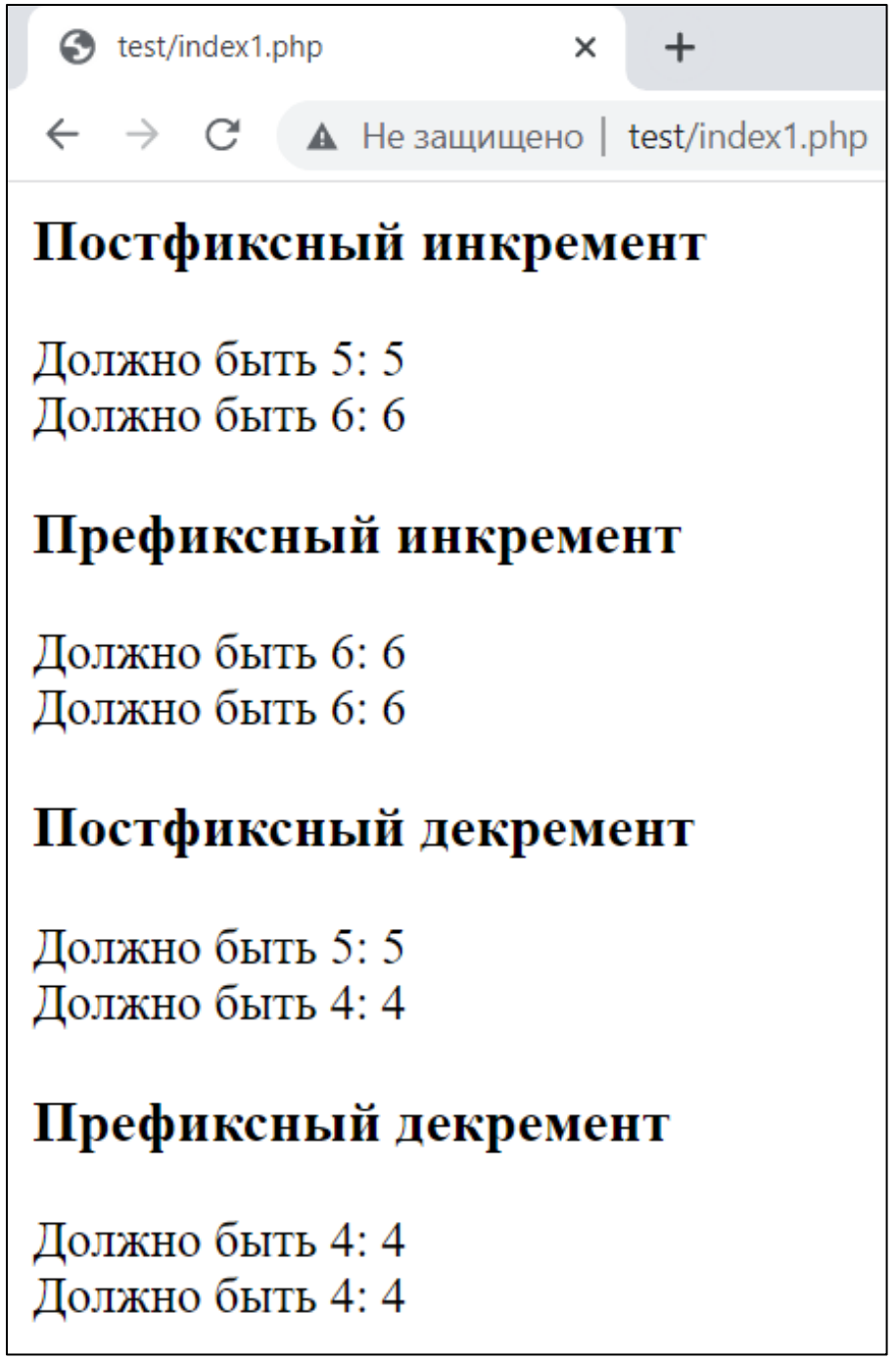
Здесь сначала значение переменной $a уменьшается на единицу, а затем ее значение присваивается переменной $b.

```php
<?php
  echo "<h3>Постфиксный инкремент</h3>";
  $a = 5;
  echo "Должно быть 5: " . $a++ . "<br>";
  echo "Должно быть 6: " . $a . "<br>";

  echo "<h3>Префиксный инкремент</h3>";
  $a = 5;
  echo "Должно быть 6: " . ++$a . "<br>";
  echo "Должно быть 6: " . $a . "<br>";

  echo "<h3>Постфиксный декремент</h3>";
  $a = 5;
  echo "Должно быть 5: " . $a-- . "<br>";
  echo "Должно быть 4: " . $a . "<br>";

  echo "<h3>Префиксный декремент</h3>";
  $a = 5;
  echo "Должно быть 4: " . --$a . "<br>";
  echo "Должно быть 4: " . $a . "<br>";
?>
```



Операции инкремента с символьными переменными

Операции инкремента можно применять и для символьных переменных. PHP следует соглашениям Perl (в отличие от C) касательно выполнения арифметических операций с символьными переменными.

Например, в PHP и Perl

```
$a = 'Z';
$a++;
```

присвоит $a значение 'AA', в то время как в C

```
a = 'Z';
a++;
```

присвоит a значение '[' (ASCII-значение 'Z' равно 90, а ASCII-значение '[' равно 91).

Операции инкремента с символьными переменными

Данная логика выполнения инкремента для символьных переменных позволяет без дополнительных операций «сдвигать» последний символ в строке к следующему по алфавиту символу.

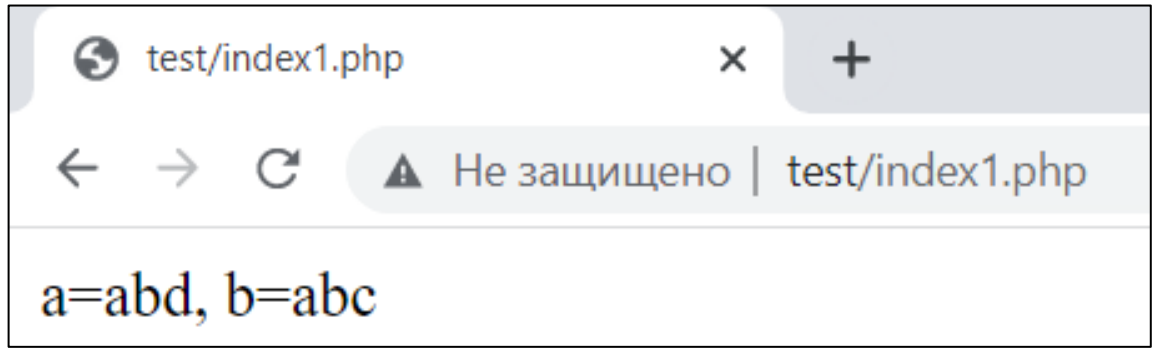
Например:

```
$a = 'ABC';
$a++;
```

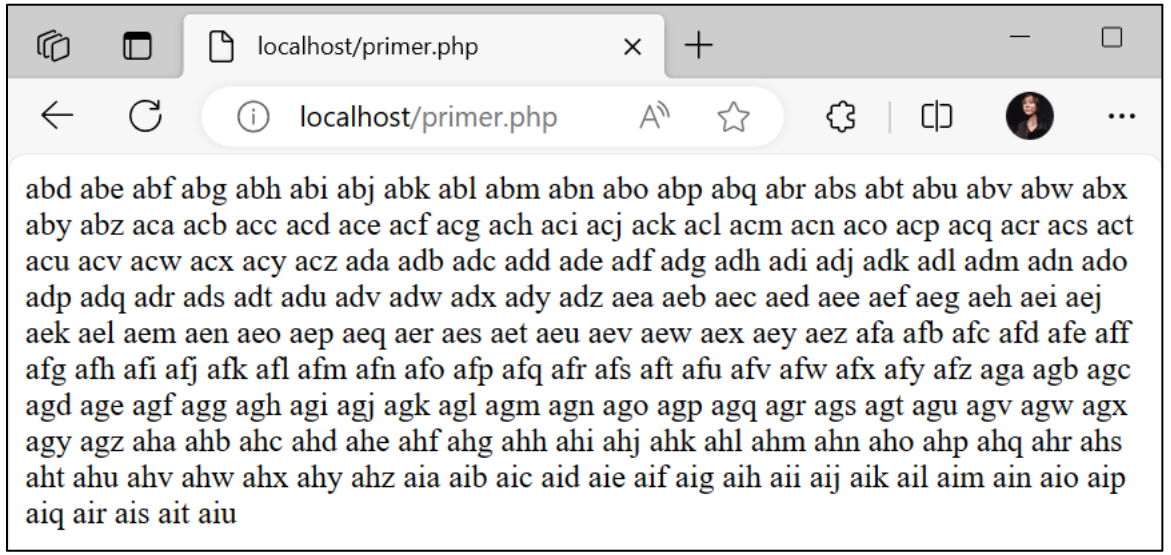
присвоит $a значение ’ABD’.

Операции инкремента с символьными переменными

```php
<?php
$a='abc';
$b=$a++;
echo "a=$a, b=$b";
// Выводит a=abd, b=abc
?>
```



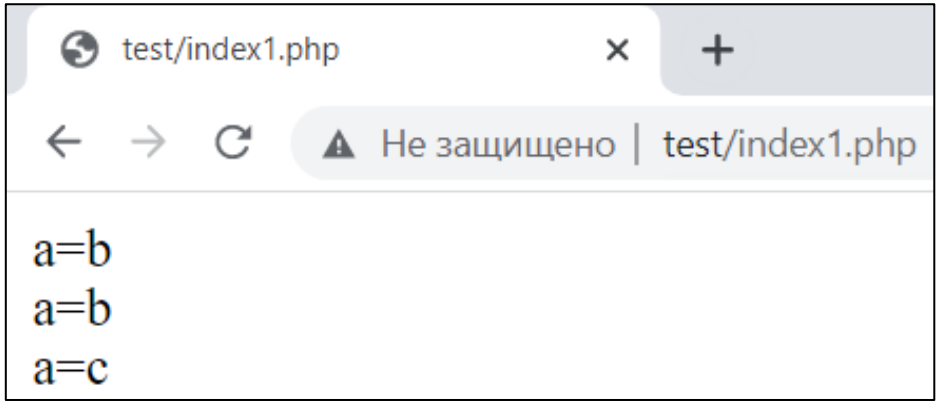

```php
<?php
$a='abc';
for ($i =0; $i <200; $i++)
{
    $a++;
    echo $a.' ';
}
```



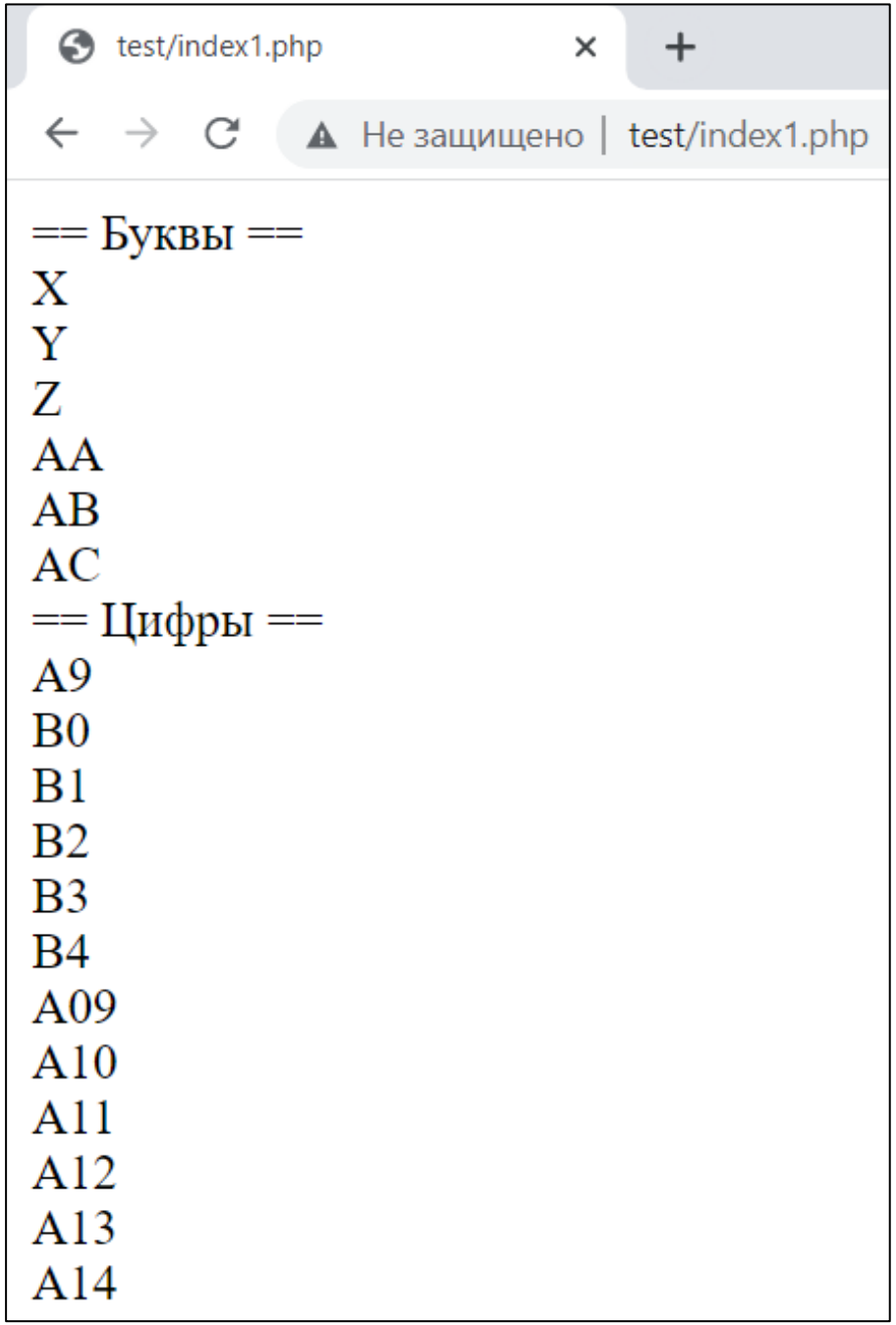
Операции инкремента с символьными переменными

С символьными переменными можно использовать как постфиксный, так и префиксный инкремент.

```php
<?php
    $a='a';
    echo 'a='.++$a,"<br>";
// Выводит a=b
    echo 'a='.$a++,"<br>";
// Выводит a=b, а затем a увеличивает на 1
    echo 'a='.$a;
?>
```



```php
<?php
echo '== Буквы ==' . "<br>";
$s = 'W';
for ($n=0; $n<6; $n++) {
    echo ++$s . "<br>";
}
// С цифрами несколько по-другому
echo '== Цифры ==' . "<br>";
$d = 'A8';
for ($n=0; $n<6; $n++) {
    echo ++$d . "<br>";
}
$d = 'A08';
for ($n=0; $n<6; $n++) {
    echo ++$d . "<br>";
}
?>
```



Операции инкремента с символьными переменными

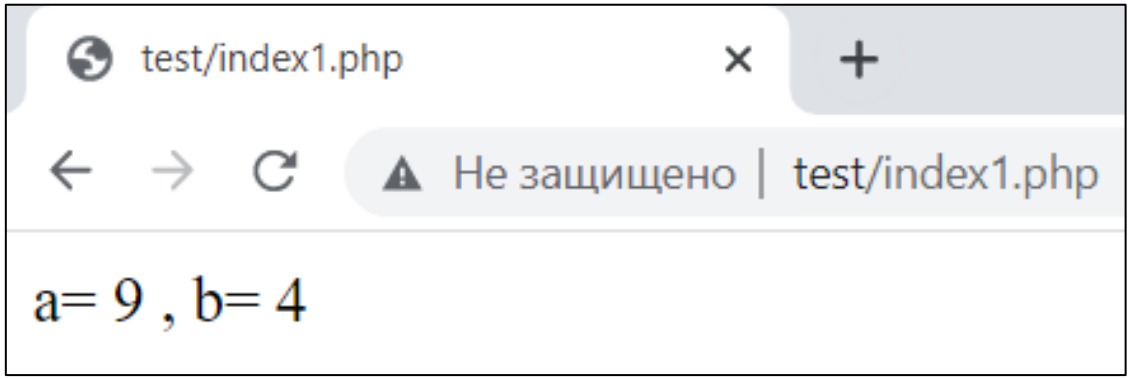


Булевые типы не подлежат инкрементированию и декрементированию.

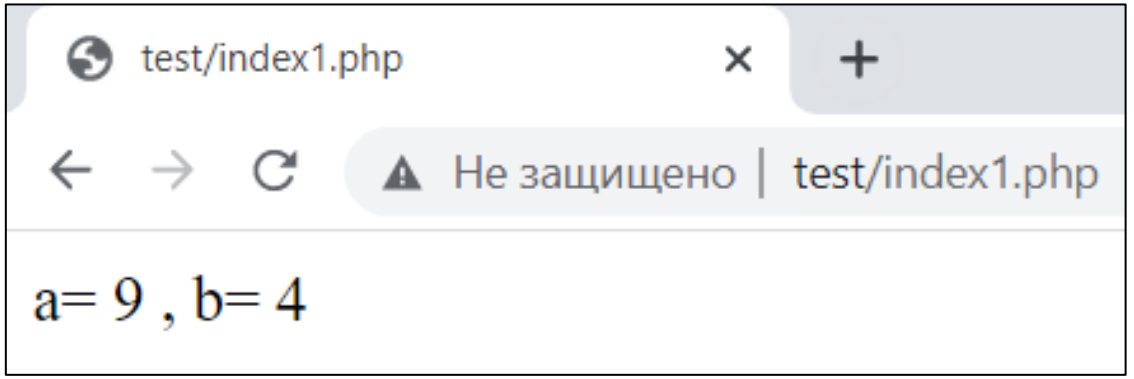
Символьный тип подлежит только операции инкрементирования (декрементирование невозможено). Кроме того поддерживается только алфавит ASCII и цифры (a-z, A-Z и 0-9).

Операции присвоения

```php
<?php
$b = 4;
$a = $b  + 5; // результат: $a установлена значением 9, переменной $b присвоено 4.
echo "a= $a , b= $b";
?>
```

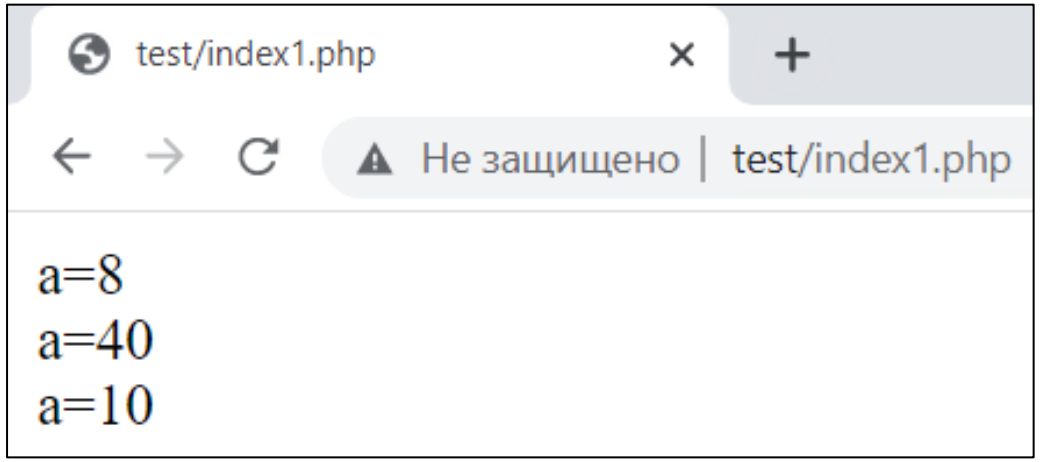



```php
<?php
$a = ($b=4)  + 5; // результат: $a установлена значением 9, переменной $b присвоено 4.
echo "a= $a , b= $b";
?>
```



Комбинирование арифметических операций и присвоения

```php
<?php
$a = 3;
$a += 5; // устанавливает $a значением 8,
аналогично записи: $a = $a + 5;
echo "a=$a";
echo"<br>";
$a *= 5; // устанавливает $a значением 40,
аналогично записи: $a = $a * 5;
echo "a=$a";
echo"<br>";
$a /= 4; // устанавливает $a значением 10,
аналогично записи: $a = $a / 4;
echo "a=$a";
echo"<br>";
?>
```



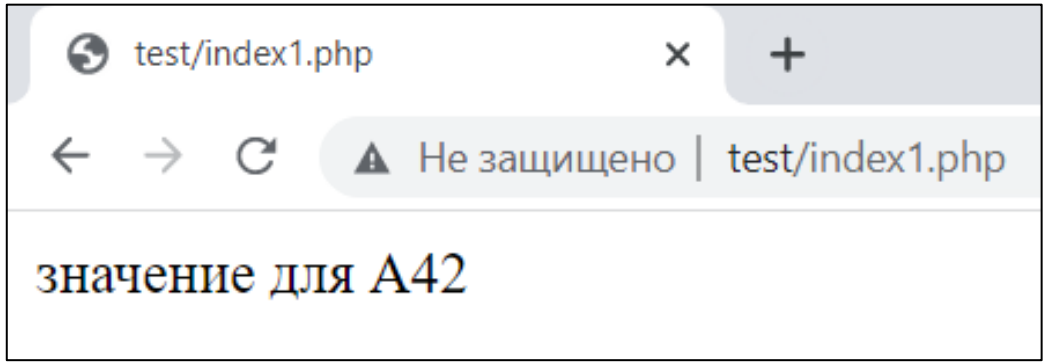
Операции присвоения

Начиная с PHP 7, добавился новый оператор «??» (null coalescing), который позволяет присваивать значение по умолчанию. Приведем простой пример.

```php
<?php
$nullValue = NULL;
$emptyText = ""; // ложноподобное
$someNumber = 42;

$valA = $nullValue ?? "значение для A";
$valB = $emptyText ?? "значение для B";
$valC = $someNumber ?? 0;

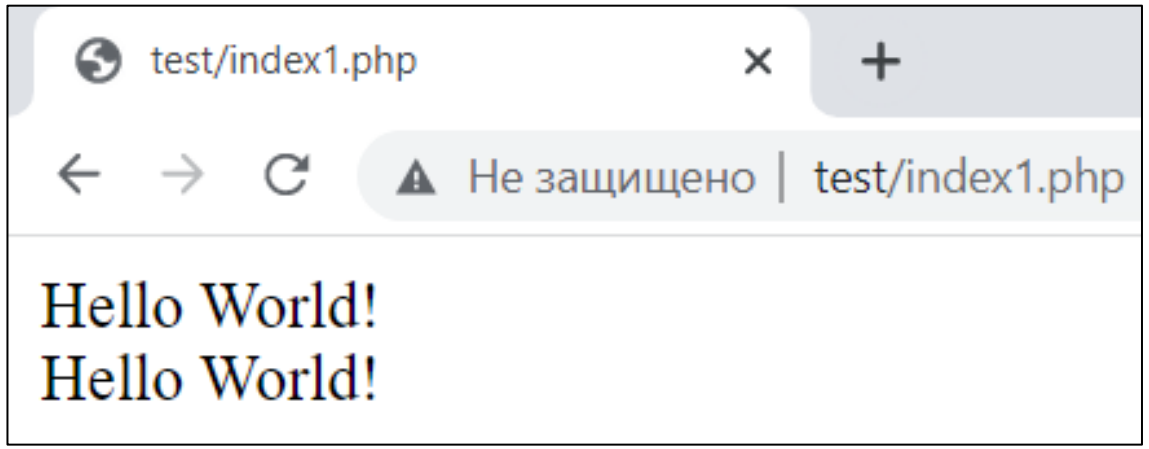
echo $valA;     // "значение для A"
echo $valB;     // "" (поскольку пустая строка не приравнивается к null или undefined)
echo $valC;     // 42
?>
```



Строковые операции

В PHP есть два оператора для работы со строками. Первый – оператор конкатенации ('.'), который возвращает объединение левого и правого аргумента. Второй – оператор присвоения с конкатенацией, который присоединяет правый аргумент к левому.

```php
<?php
$a = "Hello ";
$b = $a . "World!"; // $b содержит строку "Hello World!"
echo $b;
echo"<br>";
$a = "Hello ";
$a .= "World!";     // $a содержит строку "Hello World!"
echo $a;
?>
```



Побитовые операции

Побитовые операторы выполняют низкоуровневые манипуляции с битами в двоичных представлениях чисел. Биты целого числа — это не что иное, как отдельные разряды того же самого числа, записанного в двоичной системе счисления. **Если переменная не целая, то она в начале округляется, а уж затем к ней применяются перечисленные ниже операторы**. Для представления одного числа используются 32 бита:

0000 0000 0000 0000 0000 0000 0000 0000 - это ноль;
0000 0000 0000 0000 0000 0000 0000 0001 - это 1;
0000 0000 0000 0000 0000 0000 0000 0010 - это 2;
0000 0000 0000 0000 0000 0000 0000 0011 - это 3;
0000 0000 0000 0000 0000 0000 0000 0100 - это 4;
0000 0000 0000 0000 0000 0000 0000 0101 - это 5;
...
0000 0000 0000 0000 0000 0000 0000 1100 - это 12;
...
0000 0000 0000 0000 0000 0000 0000 1111 - это 15;
...
и т.д.

Побитовые операции

1 байт (8 бит)										
Установленное значение	**128**	**64**	**32**	**16**	**8**	**4**	**2**	**1**		
	1	1	1	1	1	1	1	1		
	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0		
	128	64	32	16	8	4	2	1	=	255

1 байт (8 бит)										
Установленное значение	**128**	**64**	**32**	**16**	**8**	**4**	**2**	**1**		
	0	1	0	1	1	1	0	1		
	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0		
	0	64	0	16	8	4	0	1	=	93

Побитовые операции

Пример	Название	Результат
$a & $b	Побитовое 'И'	Устанавливаются только те биты, которые установлены и в $a, и в $b.
$a | $b	Побитовое 'ИЛИ'	Устанавливаются те биты, которые установлены либо в $a, либо в $b.
$a ^ $b	Исключающее 'ИЛИ'	Устанавливаются только те биты, которые установлены либо только в $a, либо только в $b
~ $a	Отрицание	Устанавливаются те биты, которые в $a не установлены, и наоборот.
$a << $b	Сдвиг влево	Все биты переменной $a сдвигаются на $b позиций влево (каждая позиция подразумевает 'умножение на 2')
$a >> $b	Сдвиг вправо	Все биты переменной $a сдвигаются на $b позиций вправо (каждая позиция подразумевает 'деление на 2')

Побитовые операции

1.Побитовое «И»:

И	0011
	0101
	0001

, т.е. в десятичной форме 1.

2. Побитовое «ИЛИ»:

ИЛИ	0011
	0101
	0111

, т.е. в десятичной форме 7.

3. Побитовое исключающее «ИЛИ»:

Искл. ИЛИ	0011
	0101
	0110

, т.е. в десятичной форме 6.

Побитовые операции

4. Сдвиг влево (не забываем, что все числа в php кодируются 32 битами):

12<<2 → 0000 0000 0000 0000 0000 0000 0000 1100<<2 →

→ 0000 0000 0000 0000 0000 0000 0011 0000, т.е. в десятичной форме 48.

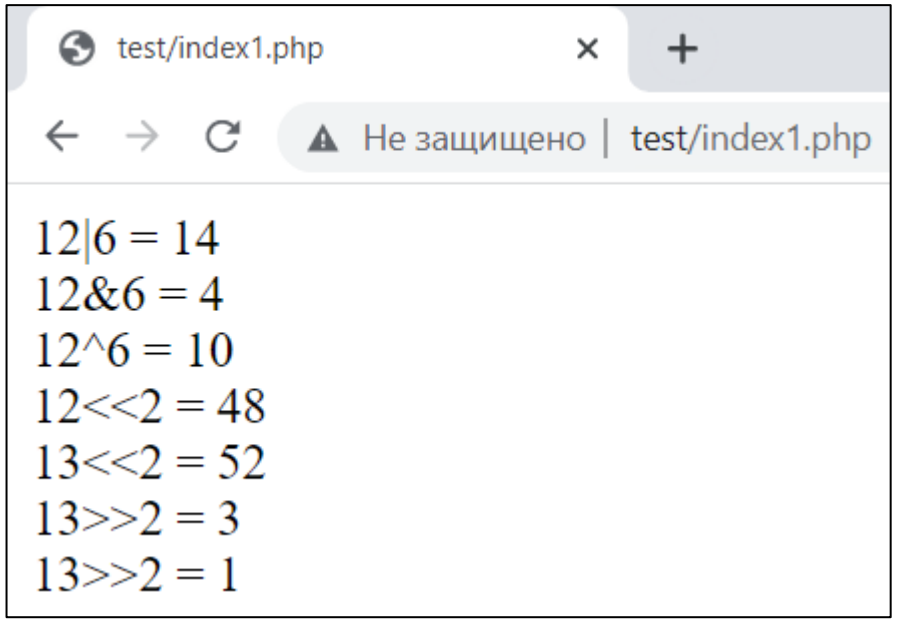
13<<2 → 0000 0000 0000 0000 0000 0000 0000 1101<<2 →

→ 0000 0000 0000 0000 0000 0000 0011 0100, т.е. в десятичной форме 52.

13>>2 → 0000 0000 0000 0000 0000 0000 0000 1101>>2 →

→ 0000 0000 0000 0000 0000 0000 0000 0011, т.е. в десятичной форме 3.

```php
<?php
$a = 12; //в двоичной системе 12 - это 1100
$b=$a|6; //в двоичной системе 6 - это  0110
echo "12|6 = $b", "<br>"; // результат = 14
$b=$a&6;
echo "12&6 = $b", "<br>";  // результат = 4
$b=$a^6;
echo "12^6 = $b", "<br>"; // результат = 10
$b=$a<<2;
echo "$a<<2 = $b", "<br>"; // результат = 48
$b=++$a<<2;
echo "$a<<2 = $b", "<br>"; // результат = 52
$b=$a>>2;
echo "$a>>2 = $b", "<br>"; // результат = 3
$b=$a>>3;
echo "$a>>2 = $b"; // результат = 1
?>
```

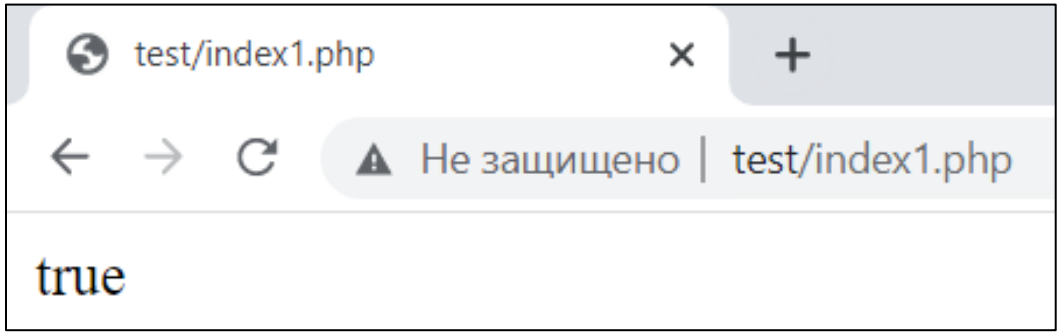


Операции сравнения

Пример	Название	Результат
$a == $b	Равно	TRUE если $a равно $b
$a === $b	Тождественно равно	TRUE если $a равно $b и имеет тот же тип
$a != $b	Не равно	TRUE если $a не равно $b
$a <> $b	Не равно	TRUE если $a не равно $b
$a !== $b	Тождественно не равно	TRUE если $a не равно $b или если они разных типов (добавлено в PHP 4 и старше)
$a < $b	Меньше	TRUE если $a строго меньше $b
$a > $b	Больше	TRUE если $a строго больше $b
$a <= $b	Меньше или равно	TRUE если $a is меньше или равно $b
$a >= $b	Больше или равно	TRUE если $a больше или равно $b
$a <=> $b	spaceship	Число типа integer меньше, больше или равное нулю, когда *$a* соответственно меньше, больше или равно *$b* (доступно с PHP 7).

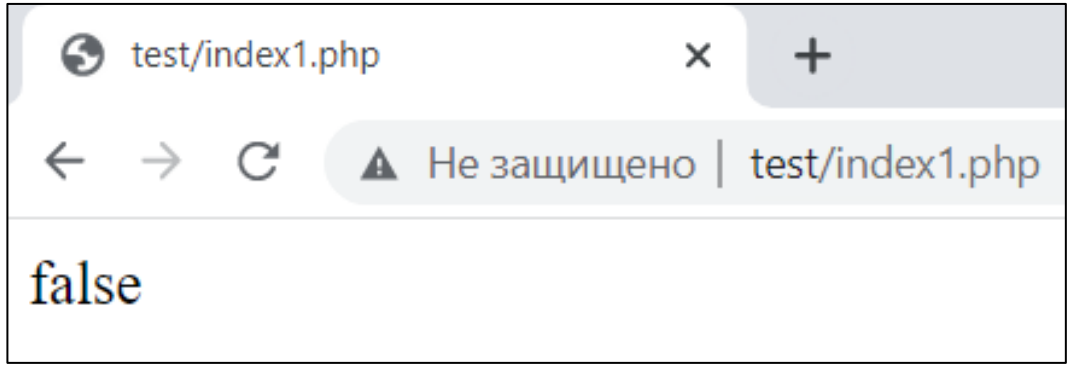
Операции сравнения

```php
<?php
$a = 10.1+0.9; //переменная типа float
$b=11;         //переменная типа integer
if ($a==$b) {
        echo 'true';
    }
    else {
        echo 'false';
}
echo"<br>";
?>
```



Операции сравнения

```php
<?php
$a = 10.1+0.9; //переменная типа float
$b=11;         //переменная типа integer
if ($a===$b) {
        echo 'true';
    }
    else {
        echo 'false';
}
echo"<br>";
?>
```

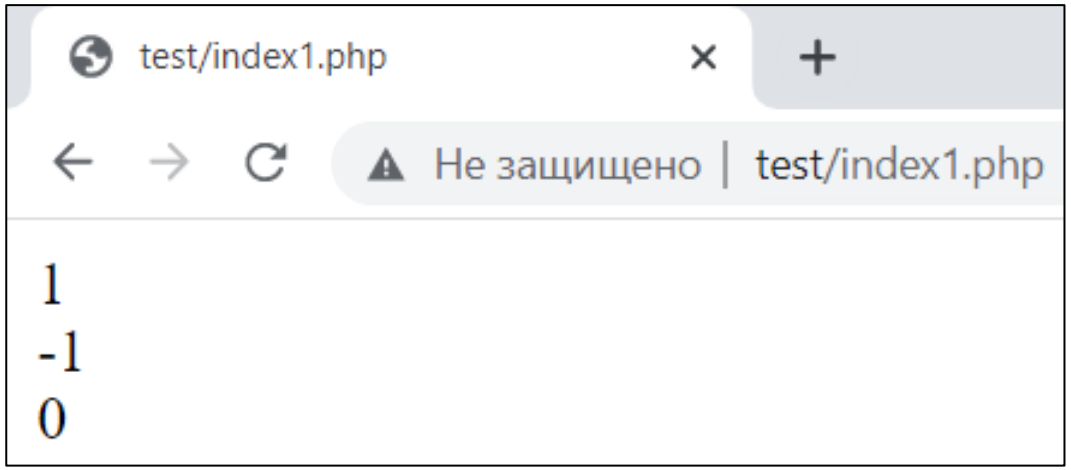


Операции сравнения

```php
<?php
$a = 12;
$b=11;
echo($a<=>$b);
echo "<br>";

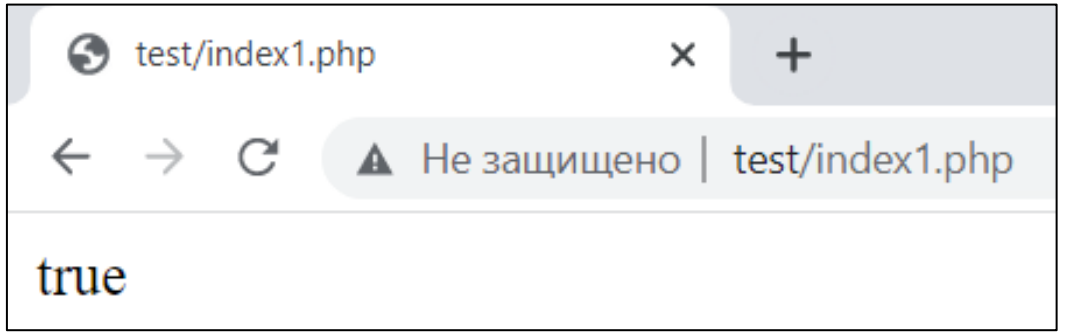
$a = 11;
$b=12;
echo($a<=>$b);
echo "<br>";

$a = 11;
$b=11;
echo($a<=>$b);
echo "<br>";
?>
```



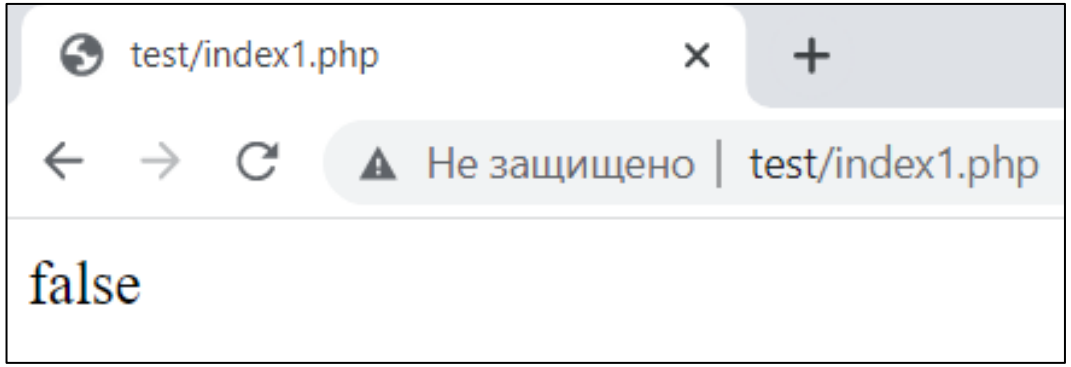
Операции сравнения

```php
<?php
$a = 0;
$b = "";
if ($a==$b) {
        echo 'true';
    }
    else {
        echo 'false';
}
echo"<br>";
?>
```



Операции сравнения

```php
<?php
$a = 0;
$b = "";
if ($a===$b) {
        echo 'true';
    }
    else {
        echo 'false';
}
echo"<br>";
?>
```



Логические операции

Пример	Название	Результат
$a and $b	Логическое 'И'	TRUE если и $a, и $b TRUE.
$a or $b	Логическое 'ИЛИ'	TRUE если или $a, или $b TRUE.
$a xor $b	Исключающее 'ИЛИ'	TRUE если $a, или $b TRUE, но не оба.
! $a	Отрицание	TRUE если $a не TRUE (инвертирует логическое значение операнда).
$a && $b	Логическое 'И'	TRUE если и $a, и $b TRUE.
$a || $b	Логическое 'ИЛИ'	TRUE если или $a, или $b TRUE.

Отметим, что “| |” имеет больший приоритет, чем “OR”, а “&&” – чем “AND”.

Логические операции



Следует заметить, что вычисление логических выражений, содержащих представленные в таблице операторы, идет всегда слева направо, при этом, если результат уже очевиден (например, *false&&что-то* всегда дает *false*), то вычисления обрываются, даже если в выражении присутствуют вызовы функций. Рассмотрим следующий пример.

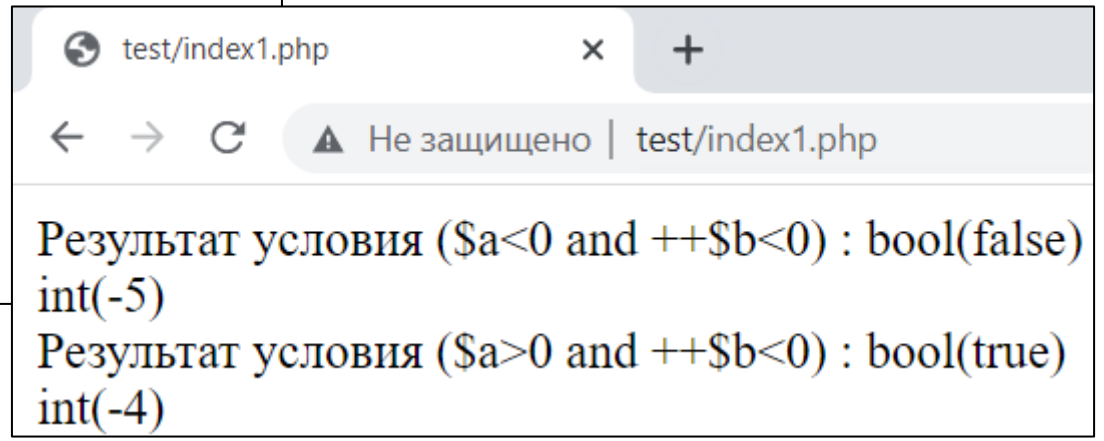
```php
<?php
    $a = 10;
    $b=-5;

    echo 'Результат условия ($a<0 and ++$b<0) : ';
    var_dump(($a<0 and ++$b<0));
    echo "<br>";

    var_dump($b);
    echo "<br>";

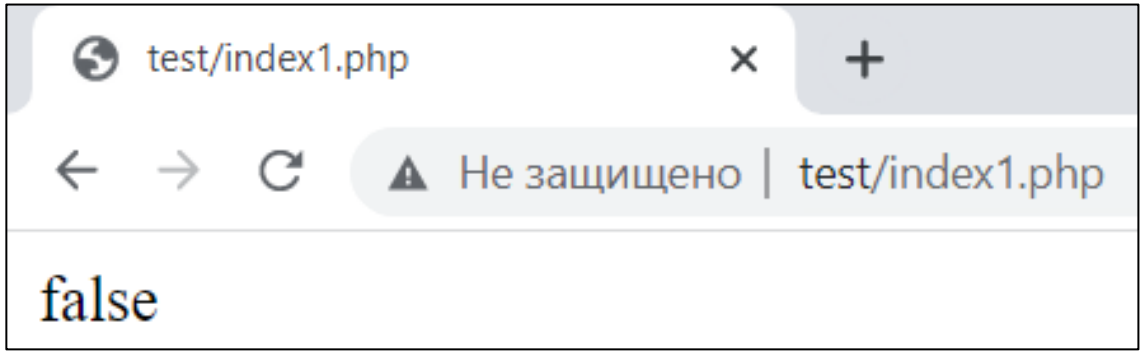
    echo 'Результат условия ($a>0 and ++$b<0) : ';
    var_dump(($a>0 and ++$b<0));
    echo "<br>";

    var_dump($b);
?>
```



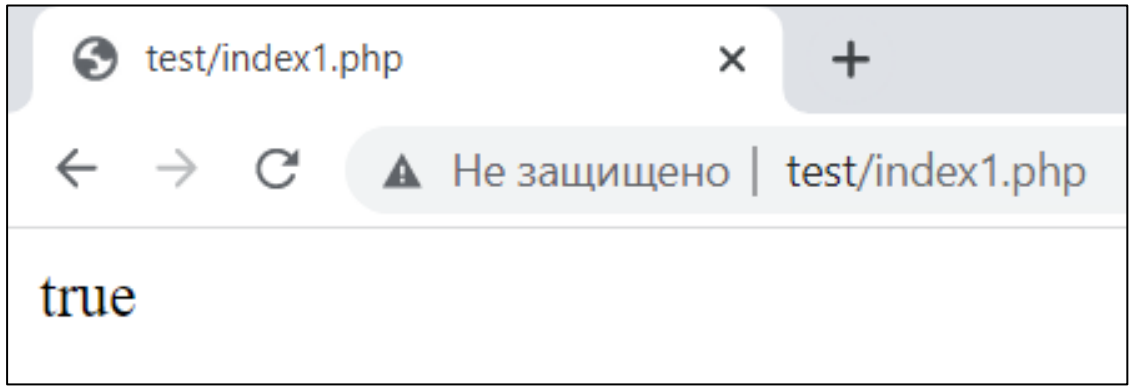
Логические операции

```php
<?php
$a = 10;
$b=-5;
if ($a>0 and $b>0)
{
    echo 'true';
} else
{
    echo 'false';
}
?>
```



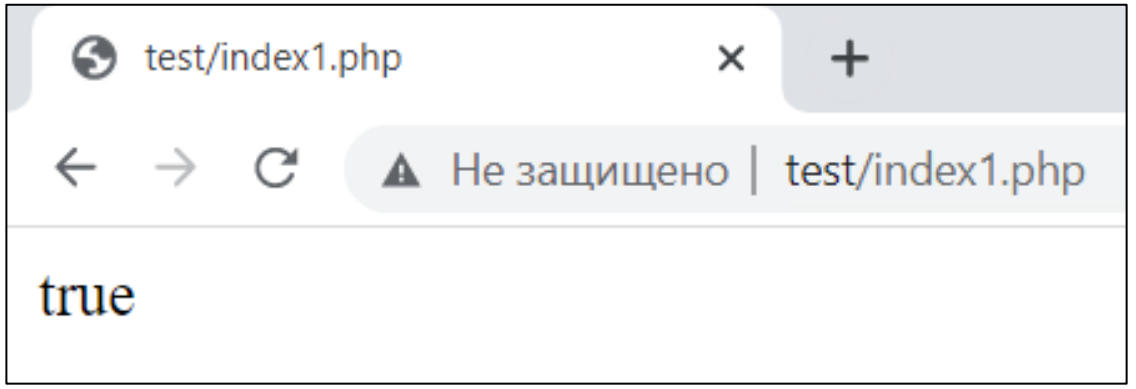
Логические операции

```php
<?php
$a = 10;
$b=-5;
if ($a>0 || $b>0)
{
    echo 'true';
} else
{
    echo 'false';
}
?>
```



Логические операции

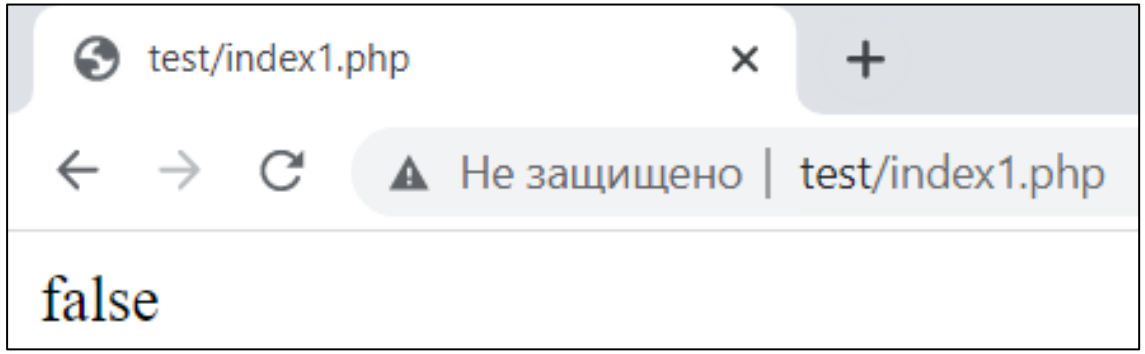
```php
<?php
$a = 10;
$b=-5;
if ($a>0 xor $b>0)
{
    echo 'true';
} else
{
    echo 'false';
}
?>
```



Оператор **исключающее ИЛИ** обозначается как XOR. Он возвращает значение true, если один и только один из операндов имеет значение true. Если оба операнда имеют значение true, оператор вернет значение false.

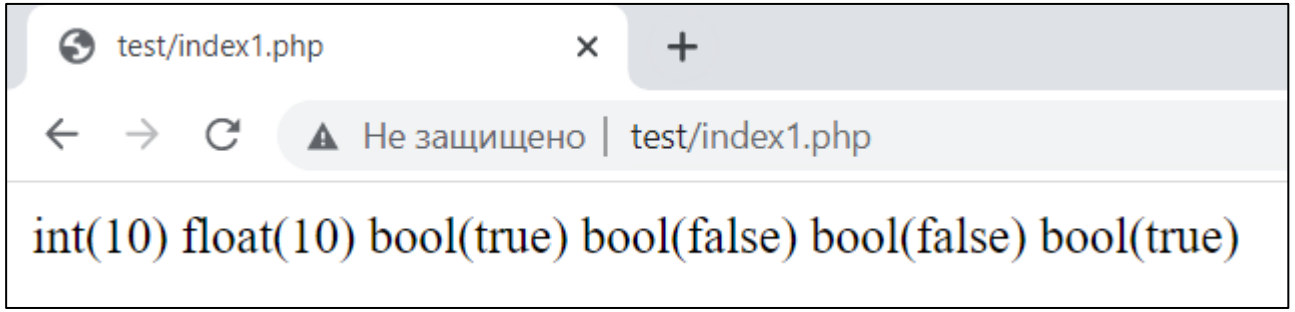
Логические операции

```php
<?php
$a = 10;
$b=5;
if ($a>0 xor $b>0)
{
    echo 'true';
} else
{
    echo 'false';
}
?>
```



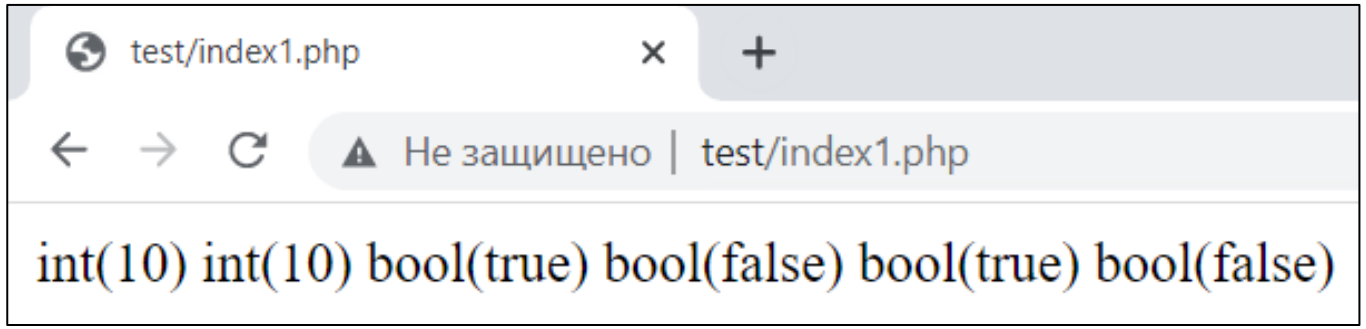
Логические операции

```php
<?php
$a = 10;
var_dump($a);
$b=55/5.5;
var_dump($b);
var_dump(($a==$b));
var_dump(!($a==$b));
var_dump(($a===$b));
var_dump(!($a===$b));
?>
```



Логические операции

```php
<?php
$a = 10;
var_dump($a);
$b=50/5;
var_dump($b);
var_dump(($a==$b));
var_dump(!($a==$b));
var_dump(($a===$b));
var_dump(!($a===$b));
?>
```



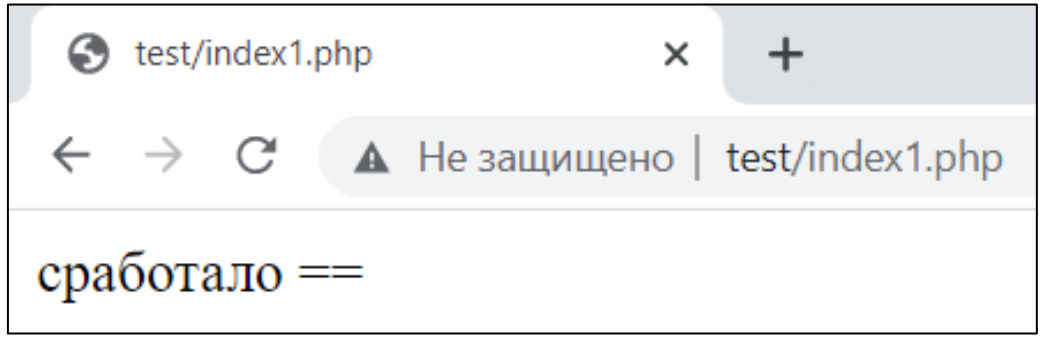
Операторы, работающие с массивами

Пример	Название	Результат
$a == $b	Равно	**TRUE** в случае, если *$a* и *$b* содержат одни и те же пары ключ/значение.
$a === $b	Тождественно равно	**TRUE** в случае, если *$a* и *$b* содержат одни и те же паты ключ/значение в том же самом порядке и того же типа.
$a != $b	Не равно	**TRUE**, если массив *$a* не равен массиву *$b*.
$a <> $b	Не равно	**TRUE**, если массив *$a* не равен массиву *$b*.
$a !== $b	Тождественно не равно	**TRUE**, если массив *$a* не равен тождественно массиву *$b*.

Операторы, работающие с массивами

```php
<?php
$a = array("apple", "banana");
$b = array(1 => "banana", 0 => "apple");

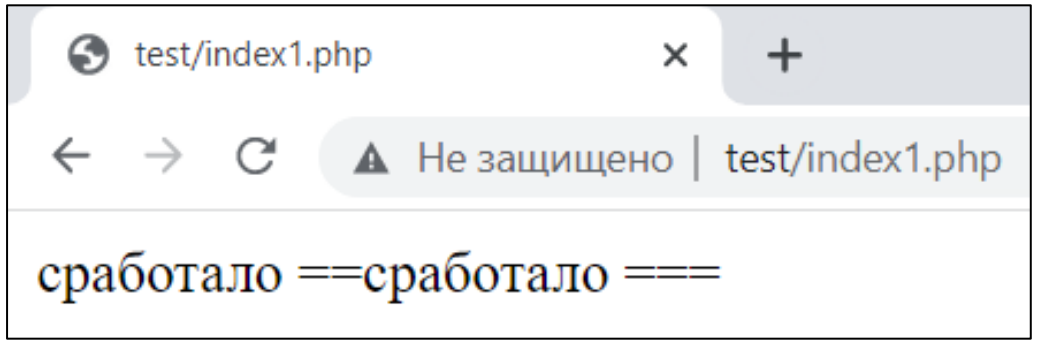
if ($a == $b) echo "сработало ==";
// bool(true)
if ($a === $b) echo "сработало ===";
// bool(false)
?>
```



Операторы, работающие с массивами

```php
<?php
$a = array("apple", "banana");
$b = array(0 => "apple", 1 => "banana");

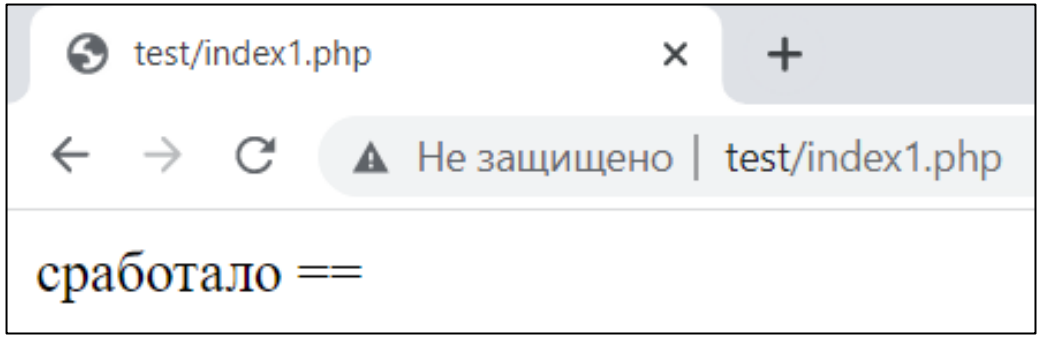
if ($a == $b) echo "сработало ==";
// bool(true)
if ($a === $b) echo "сработало ===";
// bool(false)
?>
```



Операторы, работающие с массивами

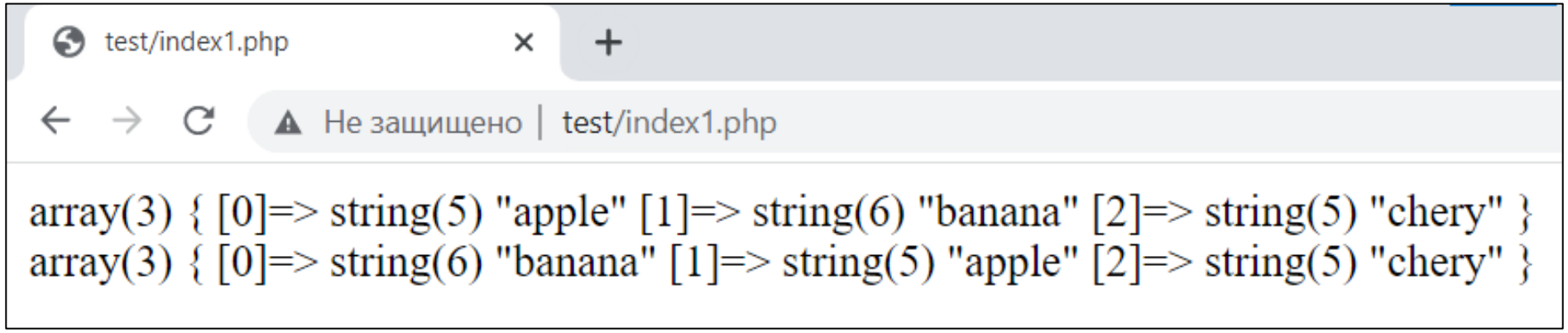
```php
<?php
$a = array(1, 2, 3);
$b = array("1", "2", "3");

if ($a == $b) echo "сработало ==";
// bool(true)
if ($a === $b) echo "сработало ===";
// bool(false)
?>
```



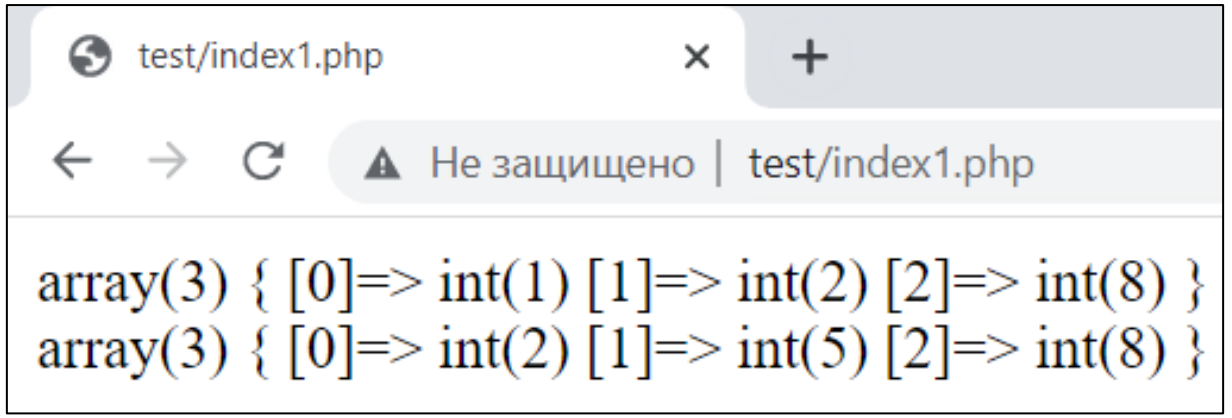
Объединение массивов

```php
<?php
$a = array("apple", "banana");
$b = array("banana", "apple", "chery");
$c=$a+$b;
var_dump($c);
echo"<br>";
$c=$b+$a;
var_dump($c);
?>
```



Объединение массивов

```php
<?php
$a = array(1, 2);
$b = array(2, 5, 8);
$c=$a+$b;
var_dump($c);
echo"<br>";
$c=$b+$a;
var_dump($c);
?>
```



Приоритеты операторов

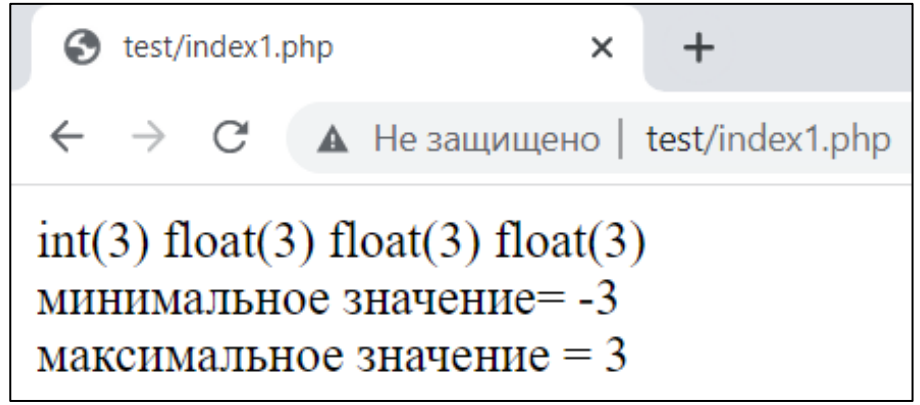
Приоритет	Оператор	Порядок выполнения
13	(постфикс)++ (постфикс)--	н/а
12	++(префикс) --(префикс)	
11	* / %	слева направо
10	+ -	слева направо
9	<< >>	слева направо
8	< <= > >=	слева направо
7	== !=	слева направо
6	&	слева направо
5	^	слева направо
4	|	слева направо
3	&&	слева направо
2	||	слева направо
1	= += -= *= /= %= >>= <<== &= ^= |=	справа налево

Простые математические функции

Функция	Описание
floor()	Принимает единственный фактический параметр (как правило, число с плавающей точкой двойной точности) и возвращает наибольшее целое число, которое меньше или равно этому фактическому параметру (округление в меньшую сторону)
ceil()	Имя этой функции представляет собой сокращение от слова ceiling (потолок). Функция принимает единственный фактический параметр (как правило, число с плавающей точкой) и возвращает наименьшее целое число, которое больше или равно этому фактическому параметру (округление в большую сторону)
round()	Принимает единственный фактический параметр (как правило, число с плавающей точкой двойной точности) и возвращает ближайшее целое число
abs()	Модуль числа. Если единственный числовой фактический параметр имеет отрицательное значение, то функция возвращает соответствующее положительное число; если фактический параметр является положительным, то функция возвращает сам фактический параметр
min()	Принимает любое количество числовых фактических параметров (но не менее одного) и возвращает наименьшее из всех значений фактических параметров
max()	Принимает любое количество числовых фактических параметров (но не менее одного) и возвращает наибольшее из всех значений фактических параметров
sqrt()	Вычисляет квадратный корень

Простые математические функции

```php
<?php
    var_dump (abs(-3));
    var_dump (round(2.7));
    var_dump (ceil(2.3));
    var_dump (floor(3.9));
    $result = min(2, abs(-3), -3,ceil(2.3) );
    echo 'минимальное значение= '.$result;
    $result = max(2, abs(-3), -3,ceil(2.3) );
    echo "<br>";
    echo 'максимальное значение = '.$result;
?>
```



Математические константы

Константа	Описание
M_PI	π
M_PI_2	π / 2
M_PI_4	π / 4
M_1_PI	1 / π
M_2_PI	2 / π
M_2_SQRTPI	2 / sqrt(π)
M_E	e
M_SQRT2	sqrt(2)
M_SQRT1_2	1 / sqrt(2)
M_LOG2E	$\log_2(e)$
M_LOG10E	lg(e)
M_LN2	$\log_e(2)$
M_LN10	$\log_e(10)$

Проверка формата чисел

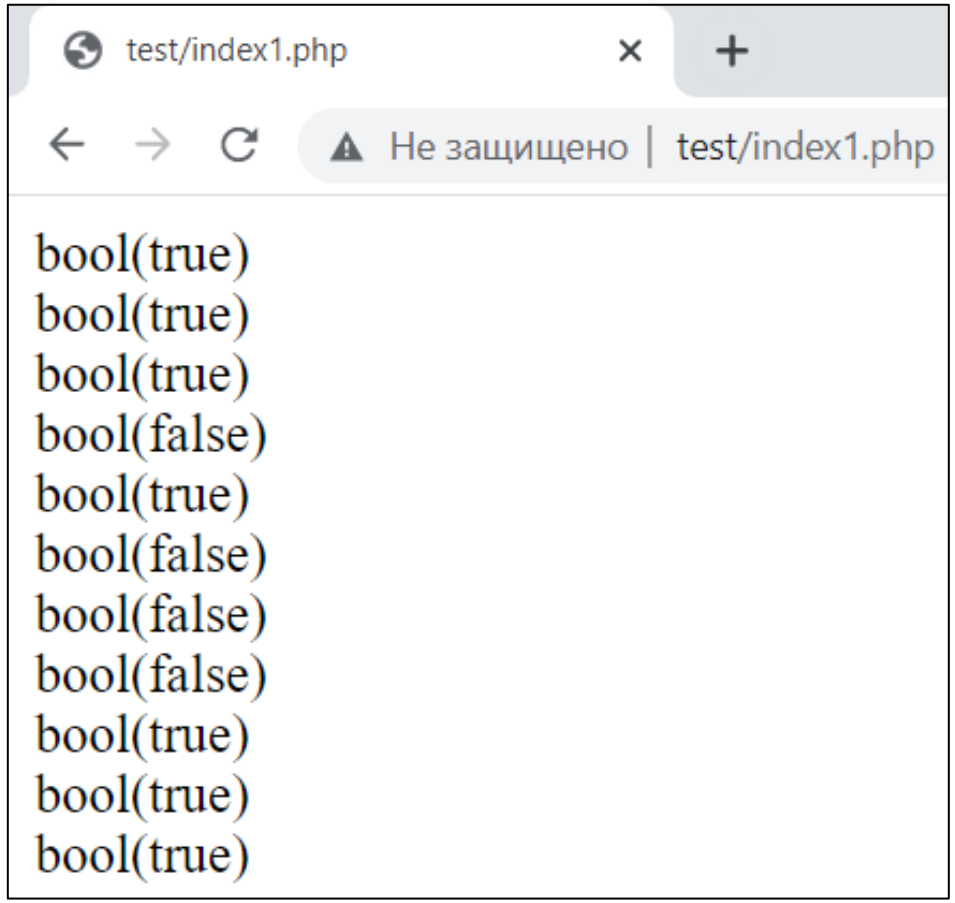
1. Первая и наиболее простая проверка заключается в использовании функции **is_numeric()**. Как и при осуществлении большинства других таких проверок, функция **is_numeric** возвращает булев результат - *true*, если переданный ей параметр представляет собой числовые данные любого типа (со знаком или без знака, целочисленные или с плавающей точкой).

2. С помощью функций **is_int()** и **is_float()** можно определить является ли число целым или дробным.

3. **is_finite()** - является ли переданная переменная (значение) допустимым конечным числом на данной платформе. **is_infinite()** возвращает TRUE, если val является бесконечностью (положительной или отрицательной), например, как результат вычисления log(0) или любые другие значения, слишком большие, чтобы уместиться в float на данной платформе.

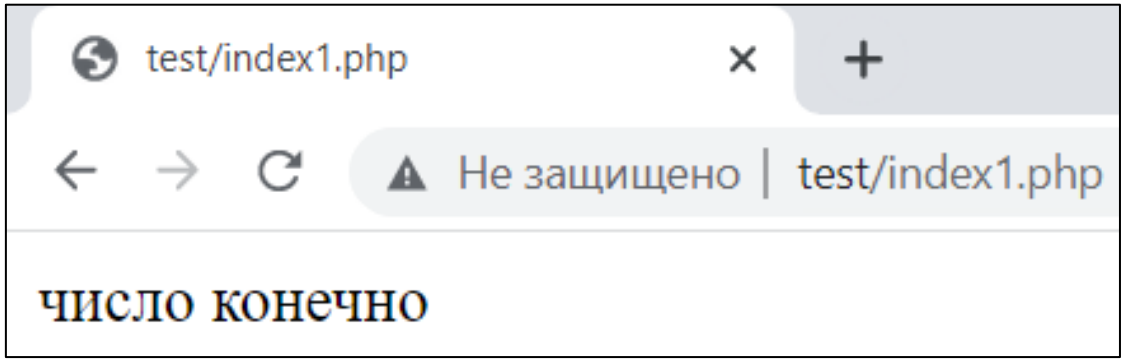
Проверка формата чисел

```php
<?php
    var_dump(is_numeric(4));          // true
    var_dump(is_numeric(25 - 6));     // true
    var_dump(is_numeric("25"));       // true
    var_dump(is_numeric("25 - 6"));   // false
    var_dump(is_int(4));              // true
    var_dump(is_int(4.2));            // false
    var_dump(is_int("4"));
    // false - данная проверка строже, чем
    проверка с помощью функции is_numeric()
    var_dump(is_float(4));            // false
    var_dump(is_float(4.0));          // true
    var_dump(is_float(M_PI));         // true
    var_dump(is_float(M_PI));         // true
?>
```

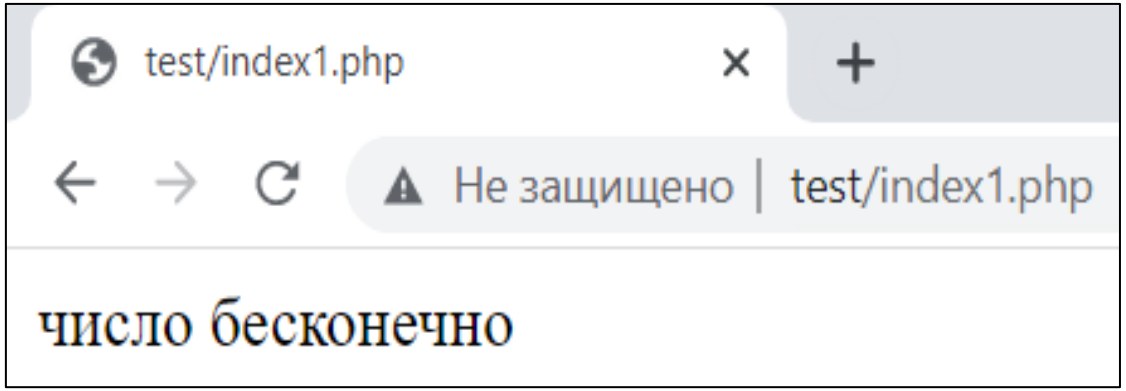


Проверка формата чисел

```php
<?php
    if (is_infinite(log(1))===true)
       {echo 'число бесконечно';}
       else
       {echo 'число конечно';}
?>
```




```php
<?php
    if (is_infinite(log(0))===true)
       {echo 'число бесконечно';}
       else
       {echo 'число конечно';}
?>
```

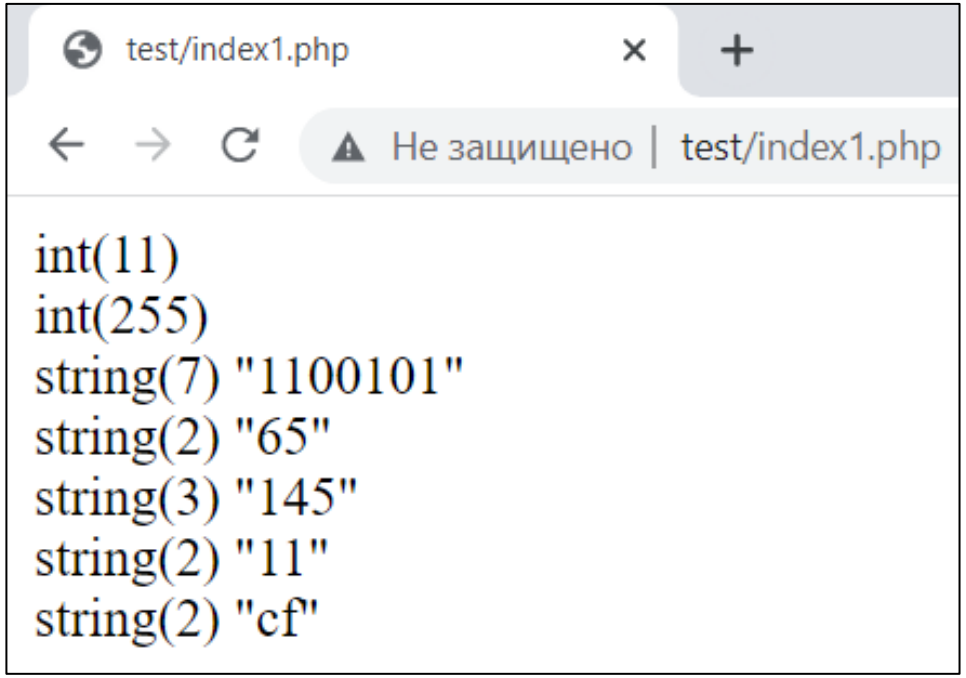


Преобразование систем счисления

Функция	Описание
BinDec()	Принимает единственный строковый параметр, представляющий собой двоичное целое число (число по основанию 2), и возвращает строковое представление этого числа по основанию системы счисления 10
DecBin()	Аналогична BinDec(), но преобразует из основания системы счисления 10 в основание системы счисления 2
OctDec()	Аналогична BinDec(), но преобразует из основания системы счисления 8 в основание системы счисления 10
DecOct()	Аналогична BinDec(), но преобразует из основания системы счисления 10 в основание системы счисления 8
HexDec()	Аналогична BinDec(), но преобразует из основания системы счисления 16 в основание системы счисления 10
DecHex()	Аналогична BinDec(), но преобразует из основания системы счисления 10 в основание системы счисления 16
base_convert()	Принимает строковый параметр (представляющий целое число, которое подлежит преобразованию) и два целочисленных параметра (исходное и желаемое основание). Возвращает строку, представляющую преобразованное число. В этой строке цифры старше, чем 9 (от 10 до 35), представлены символами a-z. И исходное, и желаемые основания должны находиться в пределах 2-36

Преобразование систем счисления

```php
<?php
    var_dump (bindec('1011'));
    var_dump (hexdec('FF'));
    var_dump (decbin('101'));
    var_dump (dechex('101'));
    var_dump (decoct('101'));
    var_dump (base_convert('1011',2,10));
    var_dump (base_convert('FF',16, 20));
?>
```

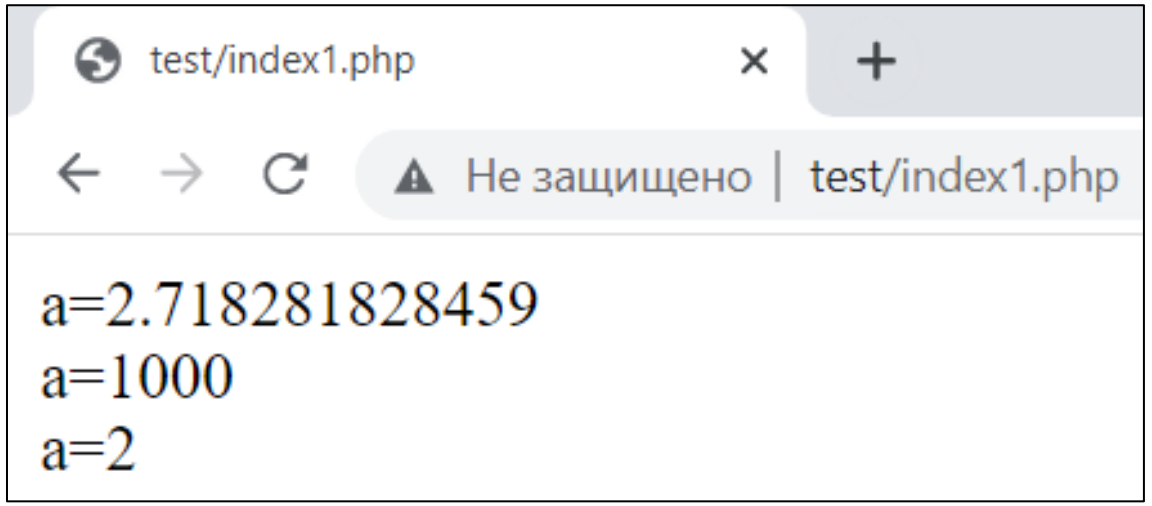


Экспоненты и логарифмы

Функция	Описание
pow()	Принимает два числовых параметра и возвращает первый параметр, возведенный в степень, равную второму параметру. Значение выражения pow($x, $y) равно x^y
exp()	Принимает единственный параметр и возводит число *e* в степень, равную этому показателю степени. Значение выражения exp($x) равно e^x
log()	Функция натурального логарифма (ln). Принимает единственный параметр и возвращает его логарифм по основанию e
log10()	Принимает единственный параметр и возвращает его десятичный логарифм (lg)

Экспоненты и логарифмы

```php
<?php
    $a=exp(1);
    echo 'a='.$a, '<br>';
    $a=pow(10,3);
    echo 'a='.$a, '<br>';
    $a=log10(100);
    echo 'a='.$a, '<br>';
?>
```



Тригонометрические функции

Функция	Описание
pi()	Не принимает параметров и возвращает приближенное значение числа π (3.1415926535898). Может использоваться как взаимозаменяемая с константой M_PI
sin()	Принимает числовой параметр в радианах и возвращает синус параметра в виде числа с плавающей точкой двойной точности
cos()	Принимает числовой параметр в радианах и возвращает косинус параметра в виде числа с плавающей точкой двойной точности
tan()	Принимает числовой параметр в радианах и возвращает тангенс параметра в виде числа с плавающей точкой двойной точности
asin()	Принимает числовой параметр и возвращает арксинус параметра в радианах. Входные данные должны находиться в пределах от -1 до 1 (получение функцией входных данных, выходящих за пределы этого диапазона, приводит к получению результата NAN). Результаты находятся в диапазоне от -π/2 до π/2
acos()	Принимает числовой параметр и возвращает арккосинус параметра в радианах. Входные данные должны находиться в пределах от -1 до 1 (получение функцией входных данных, выходящих за пределы этого диапазона, приводит к получению результата NAN. Результаты находятся в диапазоне от 0 до π
atan()	Принимает числовой параметр и возвращает арктангенс параметра в радианах. Результаты находятся в диапазоне от -π/2 до π/2
deg2rad()	Преобразует значение из градусов в радианы
rad2deg()	Преобразует значение из радианов в градусы

Выработка случайных чисел

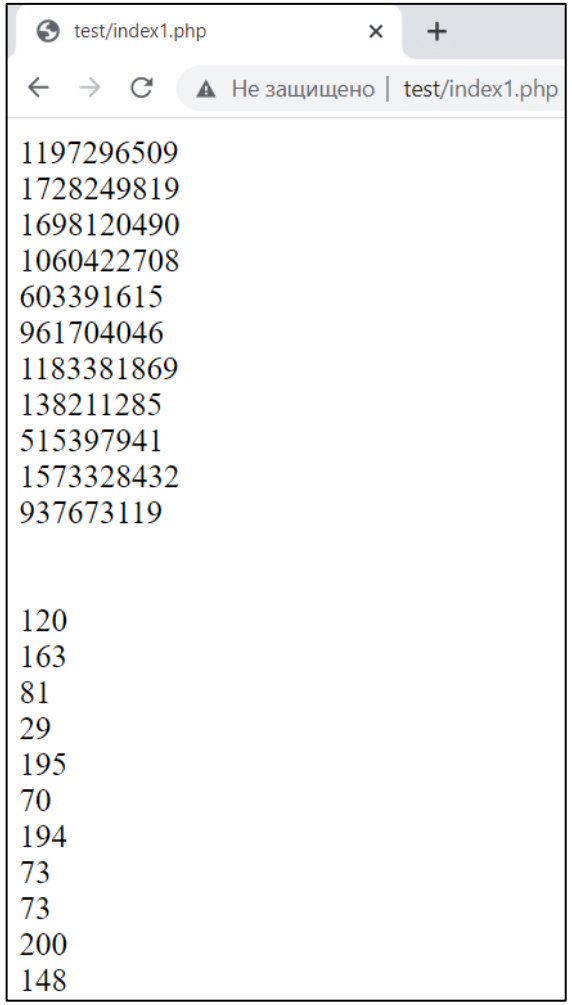
В PHP есть две группы функций по работе со случайными числами. Чисто внешне их можно отличить по префиксу **mt_** у всех функций одной из групп.
В языке PHP применяются два генератора случайных чисел - **rand()** и **mt_rand()**). С каждым из этих генераторов связаны по три функции одинакового назначения:
функция задания начального значения: **srand()** и **mt_srand()**,
сама функция получения случайного числа,
функция, осуществляющая выборку наибольшего целого числа, которое может быть возвращено генератором **getrandmax()** и **mt_getrandmax()**.

Функции getrandmax() и mt_getrandmax() возвращают значение наибольшего числа, которое может быть возвращено функцией rand() или mt_rand().

Выработка случайных чисел

```php
<?php
for ($i=0; $i<=10; $i++)
{
    echo rand();
    echo "<br>";
}
echo "<br><br>";

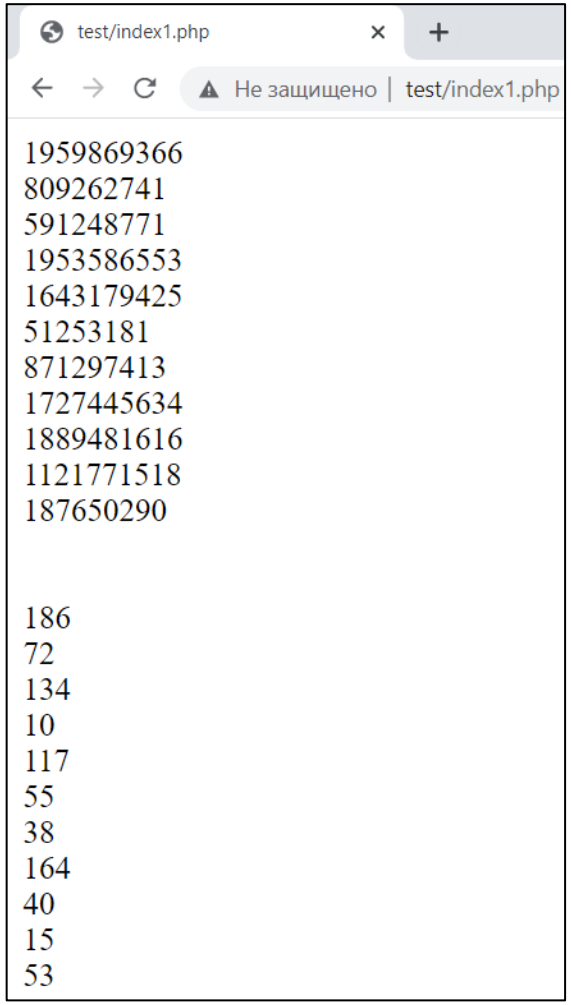
for ($i=0; $i<=10; $i++)
{
    echo rand(10, 200);
    echo "<br>";
}
?>
```



Выработка случайных чисел

```php
<?php
for ($i=0; $i<=10; $i++)
{
    echo mt_rand();
    echo "<br>";
}
echo "<br><br>";

for ($i=0; $i<=10; $i++)
{
    echo mt_rand(10, 200);
    echo "<br>";
}
?>
```



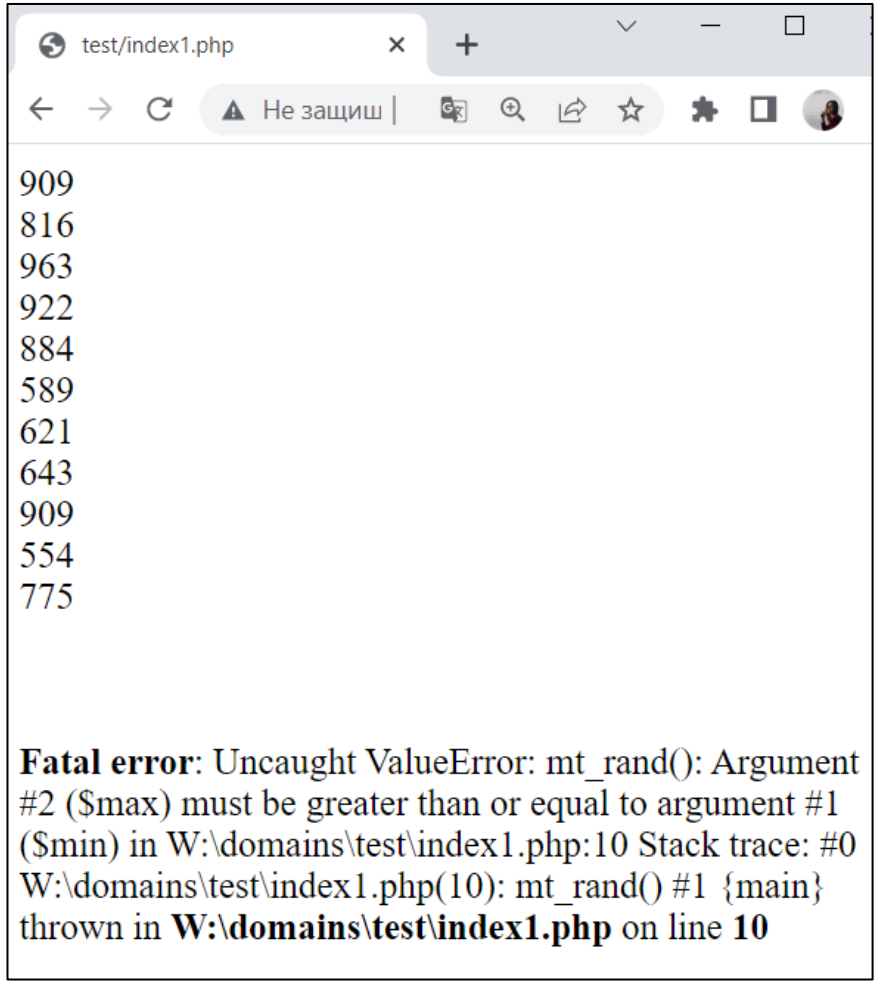
Выработка случайных чисел

Начиная с PHP 7.1.0, rand() использует тот же алгоритм получения случайных чисел, что и mt_rand(). Для сохранения обратной совместимости, функция rand() позволяет задавать параметр max меньше, чем параметр min. Функция mt_rand() в такой ситуации будет возвращать ошибку.

Выработка случайных чисел

```php
<?php
for ($i=0; $i<=10; $i++)
{
    echo rand(1000, 500);
    echo "<br>";
}
echo "<br><br>";

for ($i=0; $i<=10; $i++)
{
    echo mt_rand(1000, 500);
    echo "<br>";
}
?>
```



Выработка случайных чисел

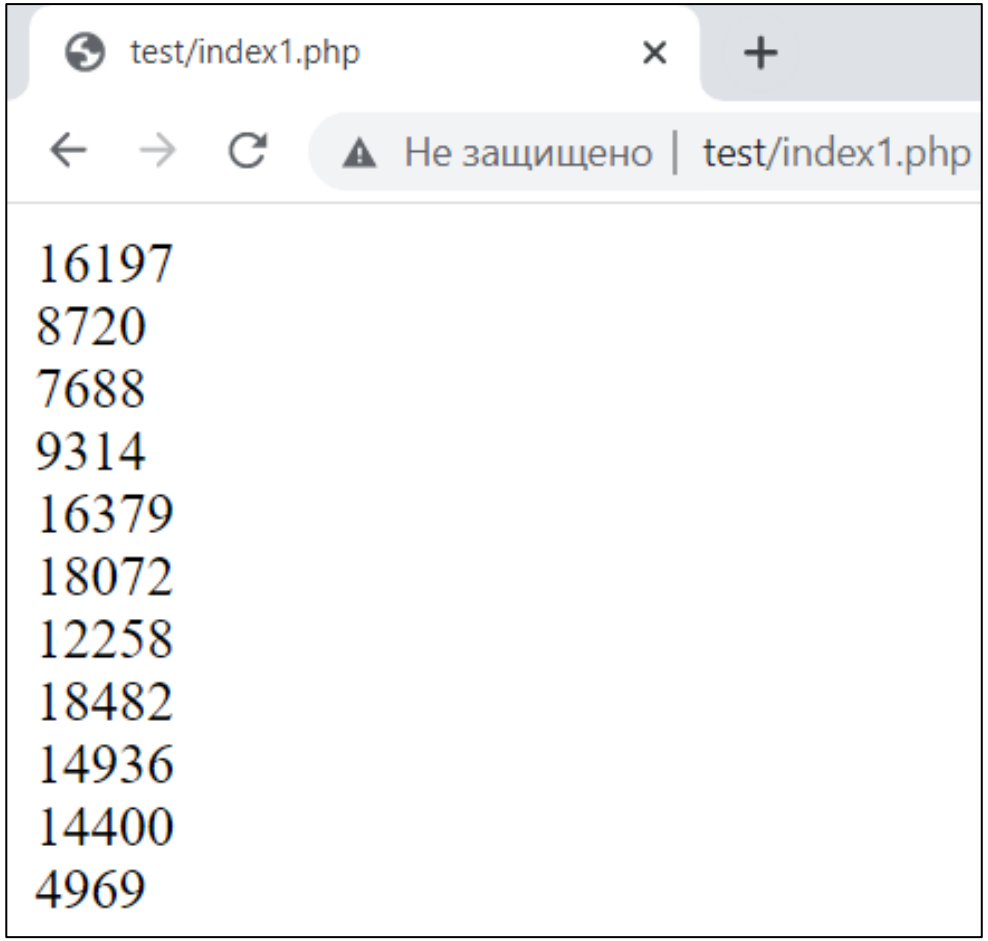
При вызове без параметров **min** и **max**, rand() и mt_rand() возвращают псевдослучайное целое в диапазоне от 0 до getrandmax() / mt_getrandmax().

Начиная с PHP версии 7.0 можно генерировать криптографически безопасные псевдослучайные целые числа (т.е., числа, для которых случайность результата очень критична). Это делается с помощью функции **random_int()**, которая также принимает параметры **min** и **max**. Эта функция более безопасна, чем функция rand(), потому что сгенерированное число непредсказуемо. Но **random_int()** медленнее, чем функция rand().

Выработка случайных чисел

```php
<?php
for ($i=0; $i<=10; $i++)
{
    echo random_int(1000, 20000);
    echo "<br>";
}
echo "<br><br>";

for ($i=0; $i<=10; $i++)
{
    echo mt_rand(10, 200);
    echo "<br>";
}
?>
```

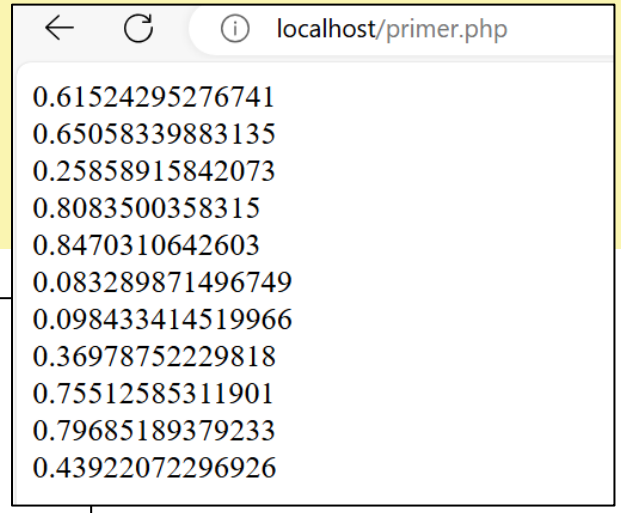


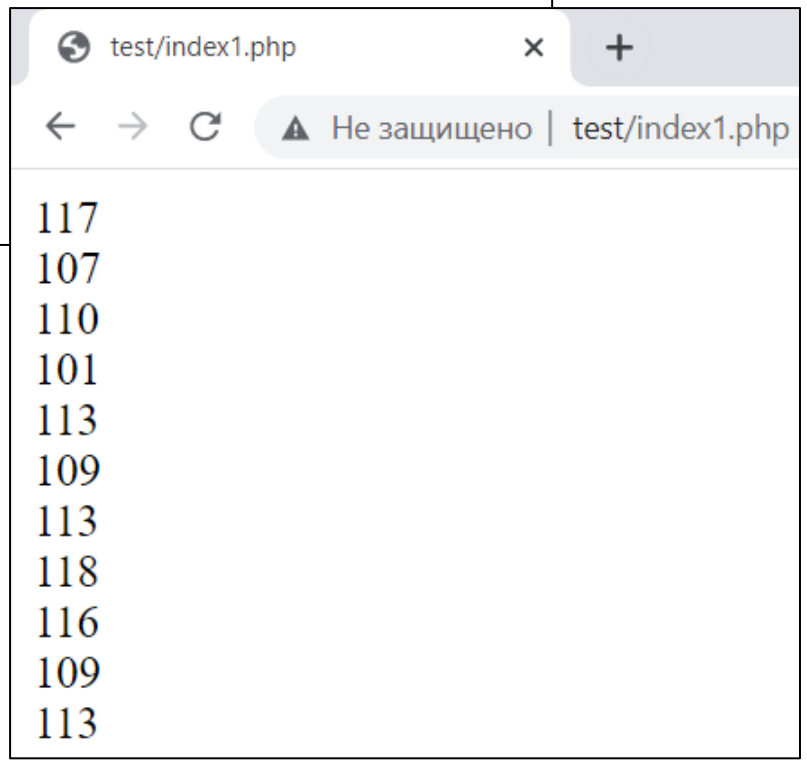
Выработка случайных чисел

Очевидно, что указанные функции выработки псевдослучайных чисел возвращают только целые числа, но случайное целое число из заданного диапазона можно легко преобразовать в соответствующее число с плавающей точкой (скажем, в число из диапазона от 0.0 до 1.0 включительно) с помощью выражения наподобие **rand() / getrandmax()**. После этого указанный диапазон можно масштабировать и сдвигать по мере необходимости.

Выработка случайных чисел

```php
<?php
for ($i=0; $i<=10; $i++){
    $random = 100.0 + 20.0 * mt_rand() / mt_getrandmax();
    echo (int)$random;
    echo "<br>";
}
?>
```

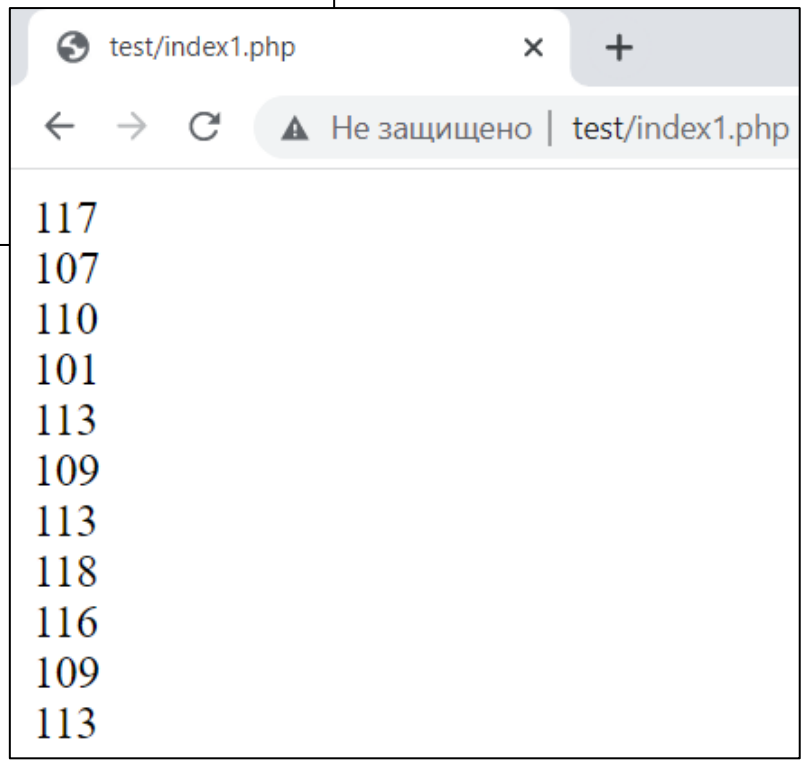




$random = a + (b-a) * случис

Выработка случайных чисел

```php
<?php
for ($i=0; $i<=10; $i++){
    $random = 100.0 + 20.0 * mt_rand() / mt_getrandmax();
    echo (int)$random;
    echo "<br>";
}
?>
```



Выработка случайных чисел

A + (B-A) * слчис(от 0 до 1)

```php
<?php
for($i=0; $i<10; $i++){
echo mt_rand() / mt_getrandmax()."<br>";
}
?>
```

